Manipuler des données avec le *tidyverse*

Bioinformatique I (BIF-7900)

Antoine Bodein

8 octobre 2026



1

INTRODUCTION

Objectifs

La semaine dernière, en R de base : `df[df$TP53 > 8, c("id", "TP53")]`

Aujourd'hui, la même chose **en mots** :

```
echantillons |>
  filter(TP53 > 8) |>
  select(id, TP53)
```

1. Importer des données et **vérifier** l'importation
2. Enchaîner des opérations avec le *pipe* `|>`
3. Sélectionner, filtrer, créer, trier : les **verbes** de `dplyr`
4. **Résumer par groupe** (`group_by()` + `summarise()`)
5. Changer la forme d'un tableau (`pivot_longer()`, `pivot_wider()`)
6. **Joindre** deux tableaux, et vérifier la jointure

Plan

1. Introduction
2. Importer avec `readr`
3. Le *pipe*
4. Les verbes de `dplyr` → **TP1**
5. Résumer par groupe
6. Des données *tidy* : `pivot_longer()` et `pivot_wider()` → **TP2**
7. Les jointures → **TP3**
8. De R de base au *tidyverse* : traduire le script de la semaine dernière

Le *tidyverse*



Une collection de *packages* qui partagent la même logique :

- `readr` : importer (`read_csv()`, `read_tsv()`)
- `dplyr` : manipuler (`filter()`, `mutate()`, `left_join()`…)
- `tidyr` : changer la forme (`pivot_longer()`…)
- `ggplot2` : visualiser (module suivant)
- `stringr`, `forcats`, `lubridate` : textes, facteurs, dates

```
library(tidyverse)   # charge les packages principaux d
```

> Un seul principe : **chaque fonction prend un tableau en premier argument et renvoie un tableau.**

R de base ou *tidyverse* ?

Les deux font la même chose. Le *tidyverse* se **lit** plus facilement, ce qui aide à vérifier.

Opération	**R de base (module 5)**	***tidyverse***
Garder des lignes	`df[df$TP53 > 8, ]`	`filter(df, TP53 > 8)`
Garder des colonnes	`df[, c("id", "TP53")]`	`select(df, id, TP53)`
Créer une colonne	`df$log <- log2(df$TP53)`	`mutate(df, log = log2(TP53))`
Trier	`df[order(df$TP53), ]`	`arrange(df, TP53)`
Moyenne par groupe	`tapply(df$TP53, df$tissu, mean)`	`summarise(group_by(df, tissu), m = mean(TP53))`

Ce qui ne change pas : les vecteurs, les types, le recyclage, les `NA` et la casse des catégories. Les pièges de la semaine dernière sont toujours là, simplement mieux cachés.

Le *tibble*

Le `data.frame` du *tidyverse* : même objet, affichage plus prudent.

```
echantillons
#> # A tibble: 6 × 5
#>   id    tissu traitement  TP53   MYC
#>   <chr> <chr> <chr>      <dbl> <dbl>
#> 1 S1    foie  ctrl         5.1   6.8
#> 2 S2    foie  trt          9.7   4.2
#> 3 S3    rein  ctrl         4.8   7.4
#> 4 S4    rein  trt         NA     5
#> 5 S5    coeur ctrl         6.2   6.1
#> 6 S6    coeur trt         10.9   3.1
```

- dimensions et **type de chaque colonne** (`<chr>`, `<dbl>`) affichés d'emblée
- seules les premières lignes sont affichées (pas de défilement de 10 000 lignes)

Installer ou charger ?

```
install.packages("tidyverse")
```

équivaut à

```
install.packages("ggplot2")
install.packages("dplyr")
install.packages("tidyr")
install.packages("readr")
install.packages("tibble")
install.packages("stringr")
...    # et une vingtaine d'autres
```

Une seule fois par ordinateur (ou par compte sur le serveur).

```
library(tidyverse)
```

équivaut à

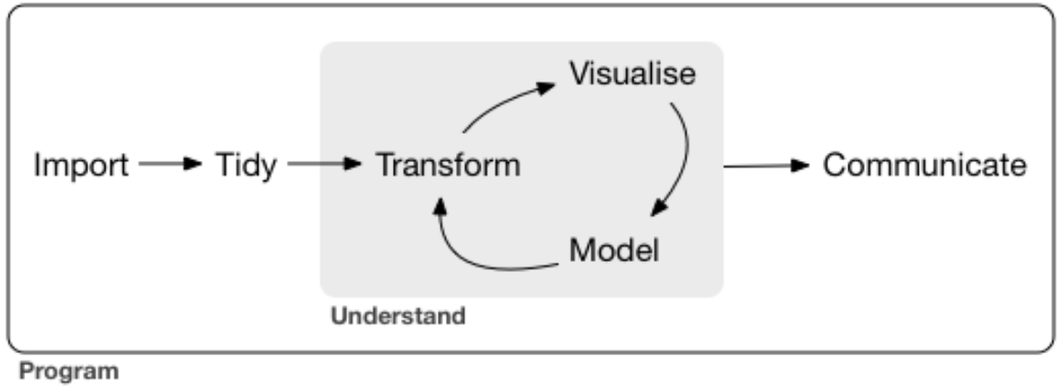
```
library(ggplot2)
library(dplyr)
library(tidyr)
library(readr)
library(tibble)
library(stringr)
library(forcats)
library(purrr)
```

À chaque session, en tête de script : un raccourci pour les packages principaux.

Le parcours d'une analyse



- **Aujourd'hui** : importer (`readr`), ranger (`tidyr`), transformer (`dplyr`)
- **Module suivant** : visualiser (`ggplot2`), qui attend des données déjà rangées et transformées



R for Data Science (H. Wickham)

2

IMPORTER AVEC `readr`



Importer un fichier texte

Les formats texte : `.csv`, `.tsv`, `.txt`...

Une ligne par **observation**, un **séparateur** entre les valeurs (ici la virgule).

Exemple : `babynames.csv`, les prénoms des bébés nés aux États-Unis de 1880 à 2017.

```
year,sex,name,n,prop
1880,F,Mary,7065,0.07238359
1880,F,Anna,2604,0.02667896
1880,F,Emma,2003,0.02052149
1880,F,Elizabeth,1939,0.01986579
1880,F,Minnie,1746,0.01788843
1880,F,Margaret,1578,0.0161672
1880,F,Ida,1472,0.01508119
1880,F,Alice,1414,0.01448696
...
```




```
/home/public/tidyverse/babynames.csv
```

Importer un fichier texte

```
bebes <- read_csv("/home/public/tidyverse/babynames.csv")
```

chaque valeur trouve sa colonne

year	sex	name	n	prop
1880	F	Mary	7065	0.07238359
1880	F	Anna	2604	0.02667896
1880	F	Emma	2003	0.02052149
1880	F	Elizabeth	1939	0.01986579
1880	F	Minnie	1746	0.01788843
1880	F	Margaret	1578	0.0161672
1880	F	Ida	1472	0.01508119
1880	F	Alice	1414	0.01448696

`read_csv()` coupe chaque ligne aux virgules, prend la première ligne comme **noms de colonnes**, et **devine** le type de chaque colonne.

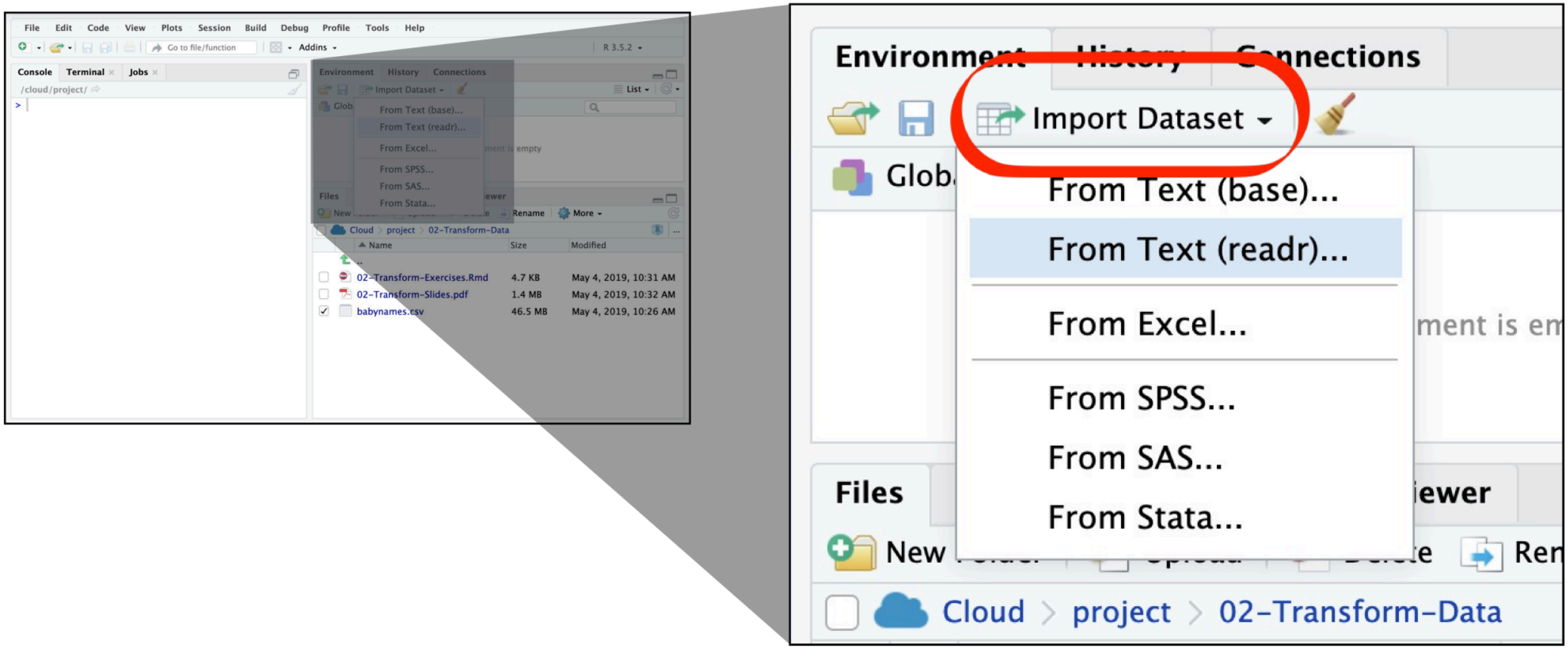
Lire un fichier en ligne de commande

Fichier	`readr`	R de base (module 5)
séparé par des virgules (`.csv`)	`read_csv("fichier.csv")`	`read.csv()`
séparé par des tabulations (`.tsv`)	`read_tsv("fichier.tsv")`	`read.table(sep = "\t", header = TRUE)`
« CSV européen » (`;` et virgule décimale)	`read_csv2("fichier.csv")`	`read.csv2()`
Excel (`.xlsx`)	`readxl::read_excel("fichier.xlsx", sheet = 1)`	

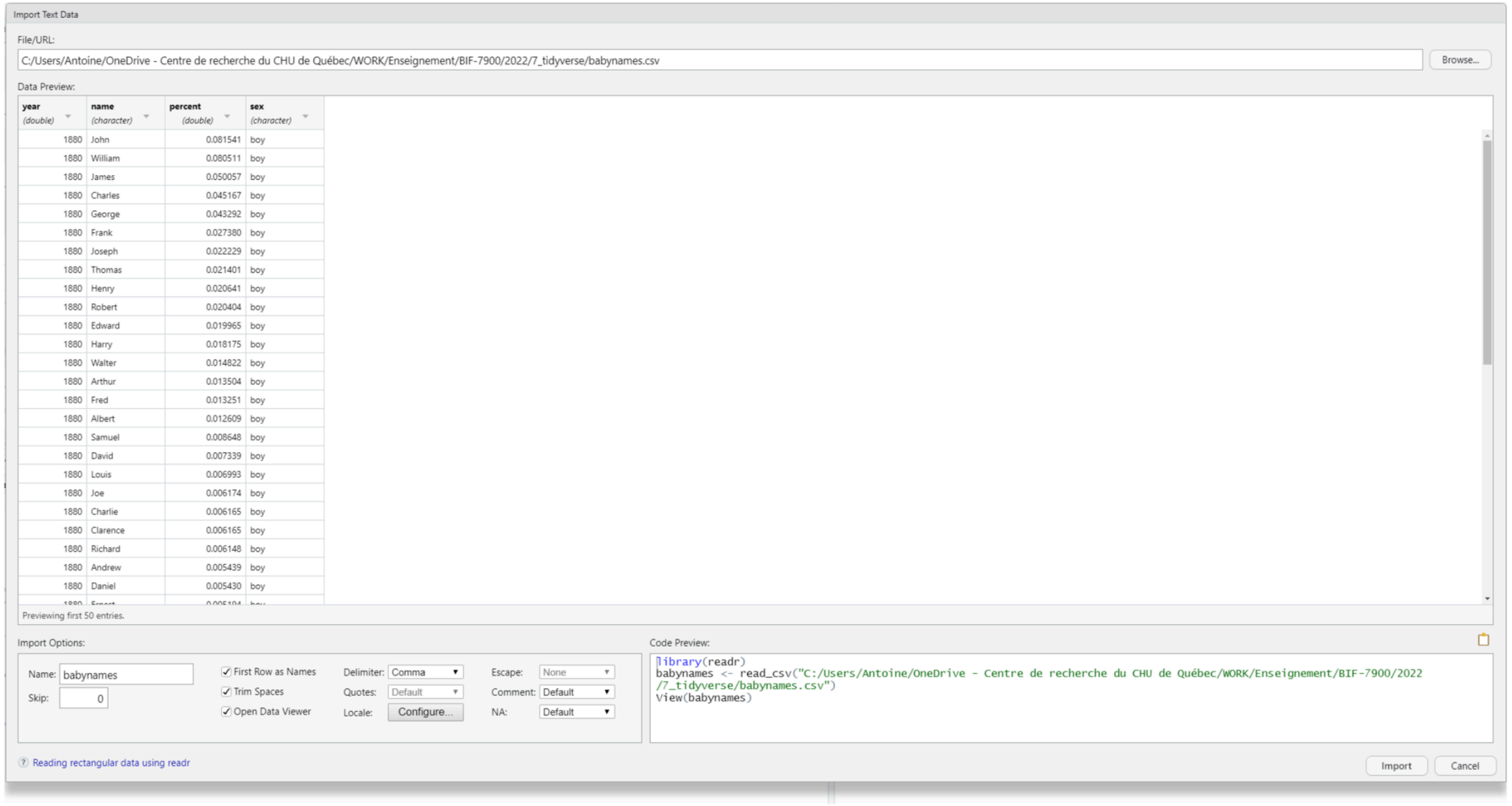
```
bebes <- read_csv("/home/public/tidyverse/babynames.csv")
```

```
#> Rows: 1924665 Columns: 5
#> ── Column specification ────────────────────────────────────────────────────
#> Delimiter: ","
#> chr (2): sex, name
#> dbl (3): year, n, prop
#>
#> ℹ Use `spec()` to retrieve the full column specification for this data.
#> ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Importer en mode interactif



Importer en mode interactif



Le *Code Preview* (en bas à droite) donne la commande `read_csv()` : **copiez-la dans votre script**, sinon l'import n'est pas reproductible.

Vérifier l'importation

`readr` **devine** le type de chaque colonne à partir de son contenu. S'il se trompe, il ne le dit pas.

Le fichier `mesures.tsv` :

```
id    TP53
S1    5.1
S2    9,7
S3    n/a
```

```
mesures <- read_tsv("mesures.tsv", show_col_types = FALSE
mesures
#> # A tibble: 3 × 2
#>   id    TP53
#>   <chr> <chr>
#> 1 S1    5.1
#> 2 S2    9,7
#> 3 S3    n/a
```

`TP53` est lue comme du **texte** (`<chr>`) : une virgule décimale et un `n/a` suffisent. Tout calcul échouera ensuite.

Réflexe : regarder les types après chaque importation (`glimpse()`, `spec()`), et préciser les valeurs manquantes : `read_tsv(..., na = c("", "NA", "n/a"))`.

Écrire un fichier

Fichier	Commande
`.csv`	`write_csv(donnees, file = "fichier.csv")`
`.tsv`	`write_tsv(donnees, file = "fichier.tsv")`
Excel, une feuille par tableau	`openxlsx::write.xlsx(list(notes = notes, iris = iris), file = "fichier.xlsx")`

- pas de numéros de rangées ni de guillemets ajoutés : le fichier se relit tel quel
- les chemins suivent les mêmes règles qu'en R de base (relatif au répertoire de travail, ou absolu)
- Excel : pratique pour échanger avec les collègues biologistes, mais **gardez le `.csv` comme fichier de travail**

3

LE *PIPE*



Une même anatomie pour toutes les fonctions

```
resultat <- fonction(tableau, options)
```

renvoie un tableau | 1er argument : un tableau | les options propres à la fonction

Puisque chaque fonction **prend** un tableau et **renvoie** un tableau, on peut les enchaîner :

```
tableau |> fonction1(...) |> fonction2(...) |> fonction3(...)
```

Le *pipe* |>



Le *pipe* prend le résultat de gauche et l'envoie comme **premier argument** de la fonction de droite.

`x |> f(y)` équivaut à `f(x, y)`. Se lit : « **puis** ».

Le même principe que le | de Bash :

```
cat babynames.csv | grep Garrett | wc -l
```



Sans *pipe* : se lit de l'intérieur vers l'extérieur

```
round(mean(c(5.1, 9.7, 4.8)), 1)
#> [1] 6.5
```

Avec *pipe* : se lit dans l'ordre d'exécution

```
c(5.1, 9.7, 4.8) |>
  mean() |>
  round(1)
#> [1] 6.5
```

`%>%` (package `magrittr`) fait la même chose : vous le verrez dans beaucoup de code et de réponses en ligne. `|>` est intégré à R depuis la version 4.1. Raccourci RStudio : **Ctrl+Maj+M**.

✋ À vous : réécrire avec le *pipe*

Réécrivez ces commandes avec `|>`, puis dites ce qu'elles renvoient.

1. `head(arrange(echantillons, MYC), 2)`
2. `nrow(filter(echantillons, tissu == "rein"))`

```
echantillons |> arrange(MYC) |> head(2)
#> # A tibble: 2 × 5
#>   id    tissu traitement  TP53   MYC
#>   <chr> <chr> <chr>      <dbl> <dbl>
#> 1 S6    coeur trt         10.9   3.1
#> 2 S2    foie  trt          9.7   4.2
echantillons |> filter(tissu == "rein") |> nrow()
#> [1] 2
```

Lisez de gauche à droite : « je prends les échantillons, **puis** je trie par MYC, **puis** je garde les 2 premiers ».

4



LES VERBES DE `dplyr`

Une grammaire de la manipulation

`dplyr` transforme un tableau avec des **verbes** :

Verbe	Rôle
`select()`	garder des **colonnes**
`filter()`	garder des **lignes**
`mutate()`	**créer** ou modifier une colonne
`arrange()`	**trier** les lignes
`summarise()`	**résumer** (souvent par groupe)



Tous s'écrivent de la même façon :

```
filter(babynames, name == "Garrett")
```

un verbe de dplyr

le tableau à transformer : toujours en premier

ce qu'il faut faire, avec les noms de colonnes sans guillemets ni $

Le résultat est **toujours un nouveau tableau** : on peut enchaîner les verbes avec `|>`.

`select()` : choisir des colonnes

```
select(babynames, name, prop)
```

babynames (extrait)

year	sex	name	n	prop
1880	M	John	9655	0.0815
1880	M	William	9532	0.0805
1880	M	James	5927	0.0501
1880	M	Charles	5348	0.0452
1880	M	Garrett	13	0.00011
1881	M	John	8769	0.081
1881	M	William	8524	0.0787
1881	M	James	5441	0.0502
1881	M	Charles	4636	0.0428
1881	M	Garrett	7	0.0000646

`select()` : choisir des colonnes

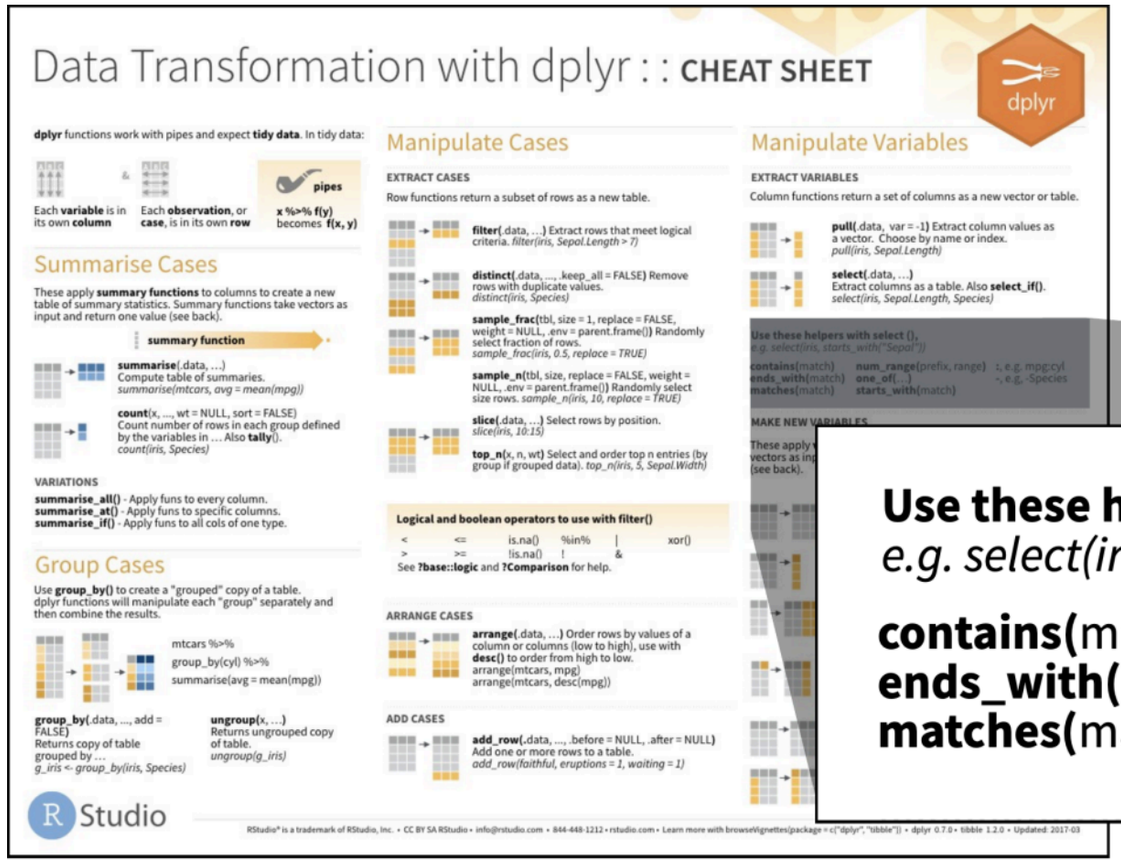
```
select(babynames, name, prop)
```

résultat : 2 colonnes

name	prop
John	0.0815
William	0.0805
James	0.0501
Charles	0.0452
Garrett	0.00011
John	0.081
William	0.0787
James	0.0502
Charles	0.0428
Garrett	0.0000646

Aides utiles : `select(-prop)` (toutes sauf), `select(year:name)` (une plage), `select(starts_with("n"))`, `select(where(is.numeric))`.

Les aides de `select()`



Use these helpers with select (),
e.g. select(iris, starts_with("Sepal"))

contains(match**)** **num_range(**prefix, range**)** **:**, e.g. mpg:cyl
ends_with(match**)** **one_of(**…**)** **-**, e.g, -Species
matches(match**)** **starts_with(**match**)**

https://raw.githubusercontent.com/rstudio/cheatsheets/main/data-transformation.pdf

`pull()` : extraire un vecteur

`select()` renvoie toujours un **tableau**, même avec une seule colonne. Pour obtenir un **vecteur**, comme avec `$` :

```
bb |> select(name) |> class()      # un tableau
#> [1] "tbl_df"     "tbl"        "data.frame"
bb |> pull(name) |> class()        # un vecteur, comme bb$name
#> [1] "character"
bb |> filter(name == "Garrett") |> pull(n)
#> [1] 13  7
```

Utile en fin de chaîne pour passer à une fonction qui attend un vecteur : `pull(n) |> sum()`, `pull(name) |> unique()`.

`filter()` : garder des lignes

```
filter(babynames, name == "Garrett")
```

babynames (extrait)

year	sex	name	n	prop
1880	M	John	9655	0.0815
1880	M	William	9532	0.0805
1880	M	James	5927	0.0501
1880	M	Charles	5348	0.0452
1880	M	Garrett	13	0.00011
1881	M	John	8769	0.081
1881	M	William	8524	0.0787
1881	M	James	5441	0.0502
1881	M	Charles	4636	0.0428
1881	M	Garrett	7	0.0000646

`filter()` : garder des lignes

```
filter(babynames, name == "Garrett")
```

le test vaut TRUE pour les lignes Garrett

year	sex	name	n	prop
1880	M	John	9655	0.0815
1880	M	William	9532	0.0805
1880	M	James	5927	0.0501
1880	M	Charles	5348	0.0452
1880	M	Garrett	13	0.00011
1881	M	John	8769	0.081
1881	M	William	8524	0.0787
1881	M	James	5441	0.0502
1881	M	Charles	4636	0.0428
1881	M	Garrett	7	0.0000646

`filter()` : garder des lignes

```
filter(babynames, name == "Garrett")
```

résultat : les lignes Garrett

year	sex	name	n	prop
1880	M	Garrett	13	0.00011
1881	M	Garrett	7	0.0000646

Plusieurs conditions : `filter(babynames, name == "Garrett", year == 1880)` (la virgule vaut **et**) : une seule ligne.

Les tests logiques

Test	Signification
`x == y`	égal à
`x != y`	différent de
`x < y`, `x <= y`	inférieur (ou égal)
`x > y`, `x >= y`	supérieur (ou égal)
`x %in% c(a, b)`	l'une des valeurs
`is.na(x)`	valeur manquante

Combiner des tests :

- `a & b`, ou `a, b` dans `filter()` : **et**
- `a | b` : **ou**
- `!a` : **non**

```
bb |> filter(name == "Garrett", year == 1880) |> pul
#> [1] 13
bb |> filter(name %in% c("John", "James"), year == 1
#> [1] 8769 5441
bb |> filter(n < 100 | name == "John") |> pull(name)
#> [1] "John"    "Garrett" "John"    "Garrett"
```

`filter()` et les `NA`

`filter(echantillons, TP53 > 6)` : mesures d'expression de 6 échantillons, dont une **manquante** (S4)

R évalue le test sur chaque ligne

id	tissu	TP53	TP53 > 6
S1	foie	5.1	FALSE
S2	foie	9.7	TRUE
S3	rein	4.8	FALSE
S4	rein	**NA**	**NA**
S5	coeur	6.2	TRUE
S6	coeur	10.9	TRUE

`filter()` et les `NA`

```
filter(echantillons, TP53 > 6)
```

seules les lignes TRUE sont gardées

id	tissu	TP53	TP53 > 6
S1	foie	5.1	FALSE
S2	foie	9.7	TRUE
S3	rein	4.8	FALSE
~~S4~~	~~rein~~	~~**NA**~~	~~**NA**~~
S5	coeur	6.2	TRUE
S6	coeur	10.9	TRUE

S4 : `NA > 6` vaut `NA`, ni `TRUE` ni `FALSE`. `filter()` l'écarte **sans message**.

`filter()` et les `NA`

`filter(echantillons, TP53 > 6)` et `filter(echantillons, TP53 <= 6)`

TP53 > 6 : 3 lignes

id	tissu	TP53	TP53 > 6
S2	foie	9.7	TRUE
S5	coeur	6.2	TRUE
S6	coeur	10.9	TRUE

TP53 <= 6 : 2 lignes

id	tissu	TP53	TP53 <= 6
S1	foie	5.1	TRUE
S3	rein	4.8	TRUE

3 + 2 ≠ 6 : S4 n'est dans **aucun** des deux. Réflexe : vérifier que les parties font le tout, et chercher les manquants avec `filter(is.na(TP53))`.

`mutate()` : créer une colonne

```
mutate(babynames, percent = round(prop * 100, 2))
```

babynames (extrait)

year	sex	name	n	prop
1880	M	John	9655	0.0815
1880	M	William	9532	0.0805
1880	M	James	5927	0.0501
1880	M	Charles	5348	0.0452
1880	M	Garrett	13	0.00011
1881	M	John	8769	0.081
1881	M	William	8524	0.0787
1881	M	James	5441	0.0502
1881	M	Charles	4636	0.0428
1881	M	Garrett	7	0.0000646

`mutate()` : créer une colonne

```
mutate(babynames, percent = round(prop * 100, 2))
```

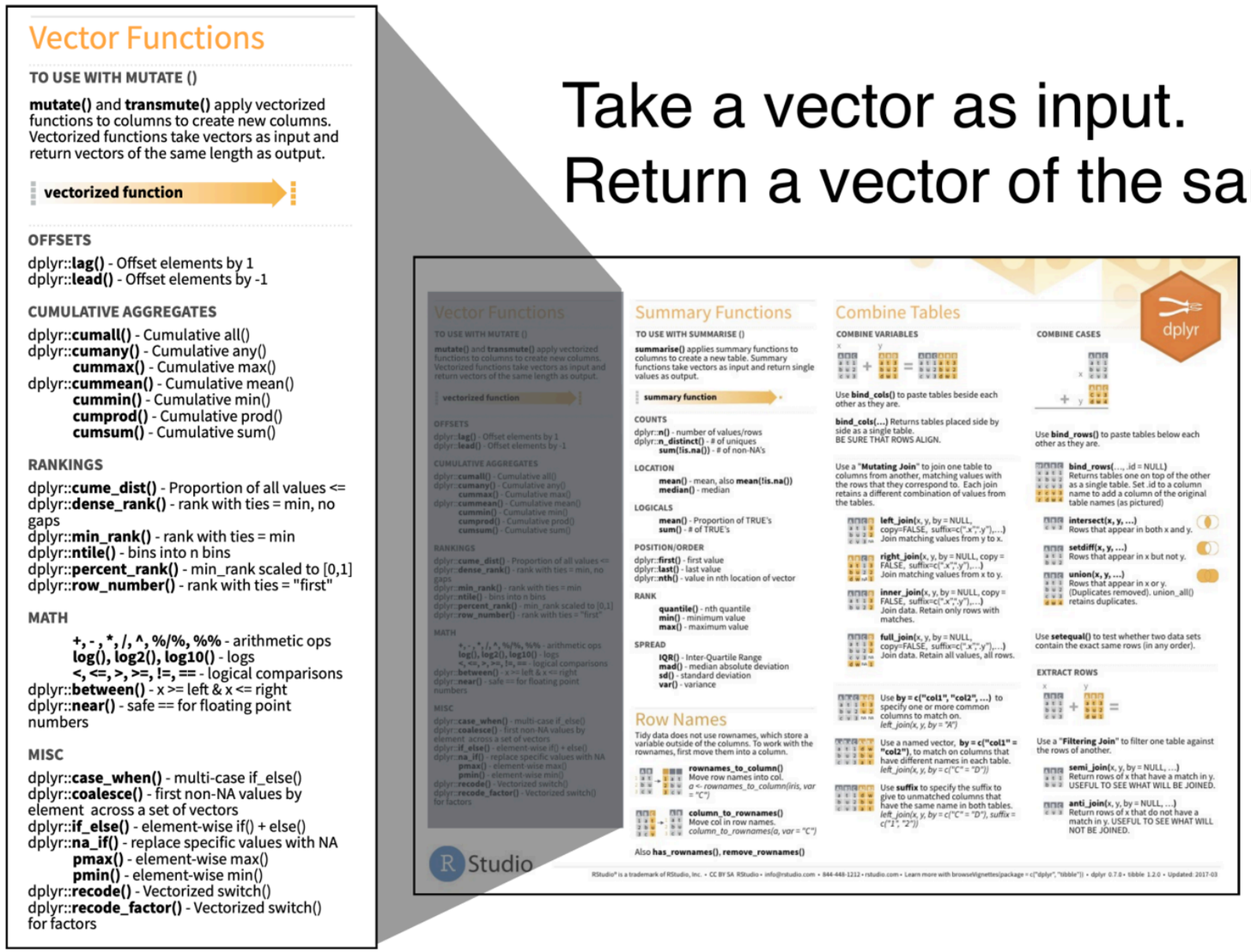
une colonne de plus, autant de lignes

year	sex	name	n	prop	percent
1880	M	John	9655	0.0815	8.15
1880	M	William	9532	0.0805	8.05
1880	M	James	5927	0.0501	5.01
1880	M	Charles	5348	0.0452	4.52
1880	M	Garrett	13	0.00011	0.01
1881	M	John	8769	0.081	8.1
1881	M	William	8524	0.0787	7.87
1881	M	James	5441	0.0502	5.02
1881	M	Charles	4636	0.0428	4.28
1881	M	Garrett	7	0.0000646	0.01

Le calcul est fait **ligne par ligne** (fonction vectorisée). Une colonne qui porte un **nom existant** est **remplacée** sans avertissement. Et rien n'est modifié tant que le résultat n'est pas **assigné** : `babynames <- babynames |> mutate(...)`.

Les fonctions vectorisées

Utilisées dans `mutate()` : elles prennent un **vecteur** et renvoient un vecteur **de même longueur**.



Vector Functions

TO USE WITH MUTATE ()

mutate() and **transmute()** apply vectorized functions to columns to create new columns. Vectorized functions take vectors as input and return vectors of the same length as output.

vectorized function

OFFSETS

dplyr::**lag()** - Offset elements by 1
dplyr::**lead()** - Offset elements by -1

CUMULATIVE AGGREGATES

dplyr::**cumall()** - Cumulative all()
dplyr::**cumany()** - Cumulative any()
cummax() - Cumulative max()
dplyr::**cummean()** - Cumulative mean()
cummin() - Cumulative min()
cumprod() - Cumulative prod()
cumsum() - Cumulative sum()

RANKINGS

dplyr::**cume_dist()** - Proportion of all values <=
dplyr::**dense_rank()** - rank with ties = min, no gaps
dplyr::**min_rank()** - rank with ties = min
dplyr::**ntile()** - bins into n bins
dplyr::**percent_rank()** - min_rank scaled to [0,1]
dplyr::**row_number()** - rank with ties = "first"

MATH

+, - , *, /, ^, %/%, %% - arithmetic ops
log(), log2(), log10() - logs
<, <=, >, >=, !=, == - logical comparisons
dplyr::**between()** - x >= left & x <= right
dplyr::**near()** - safe == for floating point numbers

MISC

dplyr::**case_when()** - multi-case if_else()
dplyr::**coalesce()** - first non-NA values by element across a set of vectors
dplyr::**if_else()** - element-wise if() + else()
dplyr::**na_if()** - replace specific values with NA
pmax() - element-wise max()
pmin() - element-wise min()
dplyr::**recode()** - Vectorized switch()
dplyr::**recode_factor()** - Vectorized switch() for factors

Take a vector as input.
Return a vector of the same length as output.

Vector Functions

TO USE WITH MUTATE ()

mutate() and transmute() apply vectorized functions to columns to create new columns. Vectorized functions take vectors as input and return vectors of the same length as output.

vectorized function

OFFSETS

dplyr::lag() - Offset elements by 1
dplyr::lead() - Offset elements by -1

CUMULATIVE AGGREGATES

dplyr::cumall() - Cumulative all()
dplyr::cumany() - Cumulative any()
cummax() - Cumulative max()
dplyr::cummean() - Cumulative mean()
cummin() - Cumulative min()
cumprod() - Cumulative prod()
cumsum() - Cumulative sum()

RANKINGS

dplyr::cume_dist() - Proportion of all values <=
dplyr::dense_rank() - rank with ties = min, no gaps
dplyr::min_rank() - rank with ties = min
dplyr::ntile() - bins into n bins
dplyr::percent_rank() - min_rank scaled to [0,1]
dplyr::row_number() - rank with ties = "first"

MATH

+, - , *, /, ^, %/%, %% - arithmetic ops
log(), log2(), log10() - logs
<, <=, >, >=, !=, == - logical comparisons
dplyr::between() - x >= left & x <= right
dplyr::near() - safe == for floating point numbers

MISC

dplyr::case_when() - multi-case if_else()
dplyr::coalesce() - first non-NA values by element across a set of vectors
dplyr::if_else() - element-wise if() + else()
dplyr::na_if() - replace specific values with NA
pmax() - element-wise max()
pmin() - element-wise min()
dplyr::recode() - Vectorized switch()
dplyr::recode_factor() - Vectorized switch() for factors

Summary Functions

TO USE WITH SUMMARISE ()

summarise() applies summary functions to columns to create a new table. Summary functions take vectors as input and return single values as output.

summary function

COUNTS

dplyr::n() - number of values/rows
dplyr::n_distinct() - # of uniques
sum(!is.na()) - # of non-NA's

LOCATION

mean() - mean, also mean(!is.na())
median() - median

LOGICALS

mean() - Proportion of TRUE's
sum() - # of TRUE's

POSITION/ORDER

dplyr::first() - first value
dplyr::last() - last value
dplyr::nth() - value in nth location of vector

RANK

quantile() - nth quantile
min() - minimum value
max() - maximum value

SPREAD

IQR() - Inter-Quartile Range
mad() - median absolute deviation
sd() - standard deviation
var() - variance

Row Names

Tidy data does not use rownames, which store a variable outside of the columns. To work with the rownames, first move them into a column.

rownames_to_column()
Move row names into col.
a <- rownames_to_column(iris, var = "C")

column_to_rownames()
Move col in row names.
column_to_rownames(a, var = "C")

Also has_rownames(), remove_rownames()

Combine Tables

COMBINE VARIABLES

Use bind_cols() to paste tables beside each other as they are.

bind_cols(...) Returns tables placed side by side as a single table.
BE SURE THAT ROWS ALIGN.

Use a "Mutating Join" to join one table to columns from another, matching values with the rows that they correspond to. Each join retains a different combination of values from the tables.

left_join(x, y, by = NULL, copy=FALSE, suffix=c(".x",".y"),...)
Join matching values from y to x.

right_join(x, y, by = NULL, copy = FALSE, suffix=c(".x",".y"),...)
Join matching values from x to y.

inner_join(x, y, by = NULL, copy = FALSE, suffix=c(".x",".y"),...)
Join data. Retain only rows with matches.

full_join(x, y, by = NULL, copy=FALSE, suffix=c(".x",".y"),...)
Join data. Retain all values, all rows.

Use by = c("col1", "col2", ...) to specify one or more common columns to match on.
left_join(x, y, by = "A")

Use a named vector, by = c("col1" = "col2"), to match on columns that have different names in each table.
left_join(x, y, by = c("C" = "D"))

Use suffix to specify the suffix to give to unmatched columns that have the same name in both tables.
left_join(x, y, by = c("C" = "D"), suffix = c("1", "2"))

COMBINE CASES

Use bind_rows() to paste tables below each other as they are.

bind_rows(..., .id = NULL)
Returns tables one on top of the other as a single table. Set .id to a column name to add a column of the original table names (as pictured)

intersect(x, y, ...)
Rows that appear in both x and y.

setdiff(x, y, ...)
Rows that appear in x but not y.

union(x, y, ...)
Rows that appear in x or y. (Duplicates removed). union_all() retains duplicates.

Use setequal() to test whether two data sets contain the exact same rows (in any order).

EXTRACT ROWS

Use a "Filtering Join" to filter one table against the rows of another.

semi_join(x, y, by = NULL, ...)
Return rows of x that have a match in y. USEFUL TO SEE WHAT WILL BE JOINED.

anti_join(x, y, by = NULL, ...)
Return rows of x that do not have a match in y. USEFUL TO SEE WHAT WILL NOT BE JOINED.

RStudio® is a trademark of RStudio, Inc. • CC BY SA RStudio • info@rstudio.com • 844-448-1212 • rstudio.com • Learn more with browseVignettes(package = c("dplyr", "tibble")) • dplyr 0.7.0 • tibble 1.2.0 • Updated: 2017-03

https://raw.githubusercontent.com/rstudio/cheatsheets/main/data-transformation.pdf

`arrange()` : trier les lignes

```
arrange(babynames, n)
```

babynames (extrait)

year	sex	name	n	prop
1880	M	John	9655	0.0815
1880	M	William	9532	0.0805
1880	M	James	5927	0.0501
1880	M	Charles	5348	0.0452
1880	M	Garrett	13	0.00011
1881	M	John	8769	0.081
1881	M	William	8524	0.0787
1881	M	James	5441	0.0502
1881	M	Charles	4636	0.0428
1881	M	Garrett	7	0.0000646

`arrange()` : trier les lignes

```
arrange(babynames, n)
```

du plus petit au plus grand

year	sex	name	n	prop
1881	M	Garrett	7	0.0000646
1880	M	Garrett	13	0.00011
1881	M	Charles	4636	0.0428
1880	M	Charles	5348	0.0452
1881	M	James	5441	0.0502
1880	M	James	5927	0.0501
1881	M	William	8524	0.0787
1881	M	John	8769	0.081
1880	M	William	9532	0.0805
1880	M	John	9655	0.0815

`arrange()` : trier les lignes

```
arrange(babynames, desc(n))
```

desc() : du plus grand au plus petit

year	sex	name	n	prop
1880	M	John	9655	0.0815
1880	M	William	9532	0.0805
1881	M	John	8769	0.081
1881	M	William	8524	0.0787
1880	M	James	5927	0.0501
1881	M	James	5441	0.0502
1880	M	Charles	5348	0.0452
1881	M	Charles	4636	0.0428
1880	M	Garrett	13	0.00011
1881	M	Garrett	7	0.0000646

Les `NA` sont toujours placés **à la fin**, quel que soit le sens. Pour les *n* plus grandes valeurs : `slice_max(n, n = 3)`.

Nettoyer avant d'analyser

Les pièges de la semaine dernière (casse, doublons) existent aussi dans le *tidyverse*. `count()` les révèle :

saisie

id	groupe
P1	traite
P2	Controle
P3	controle␣
P4	controle
P4	controle



count(saisie, groupe)

groupe	n
Controle	1
controle	2
controle␣	1
traite	1

Quatre catégories au lieu de deux : `Controle`, `controle` et `controle␣` (une **espace invisible** en fin de mot, rendue visible ici par ␣).

Nettoyer avant d'analyser

```
saisie |> mutate(groupe = str_to_lower(groupe),     # tout en minuscules
                 groupe = str_trim(groupe))        # retire les espaces au début et à la fin
```

casse et espaces corrigés

id	groupe
P1	traite
P2	controle
P3	controle
P4	controle



count(groupe)

groupe	n
controle	4
traite	1

Il reste un piège : P4 est saisi **deux fois** et compte double.

Nettoyer avant d'analyser

```
saisie |> mutate(...) |> distinct()      # retire les lignes identiques
```

une ligne par patient

id	groupe
P1	traite
P2	controle
P3	controle
P4	controle



count(groupe)

groupe	n
controle	3
traite	1

Réflexe : `count()` sur chaque colonne de texte, et `count(id) |> filter(n > 1)` pour les doublons, **avant** tout calcul. `distinct()` ne retire que les lignes **strictement identiques** : deux saisies d'un même patient avec des valeurs différentes restent, et c'est à vous de trancher.

✋ À vous : prédire le résultat

Avec `echantillons`, combien de lignes renvoie chaque commande ? Laquelle ne fait pas ce qu'elle semble faire ?

1. `filter(echantillons, MYC > 5)`
2. `filter(echantillons, TP53 != 9.7)`
3. `filter(echantillons, tissu == "Foie")`
4. `filter(echantillons, tissu == c("foie", "rein"))`

```
filter(echantillons, MYC > 5) |> nrow()
#> [1] 3
filter(echantillons, TP53 != 9.7) |> nrow()
#> [1] 4
filter(echantillons, tissu == "Foie") |> nrow()
#> [1] 0
filter(echantillons, tissu == c("foie", "rein")) |> pull(id)
#> [1] "S1" "S4"
```

2. S4 (`NA`) est écarté en plus de S2. **3.** La casse compte. **4.** La ligne 1 est comparée à `"foie"`, la 2 à `"rein"`, la 3 à `"foie"`… (**recyclage**) : 2 lignes au lieu de 4. Bonne écriture : `tissu %in% c("foie", "rein")`.

TP1

5



RÉSUMER PAR GROUPE

`summarise()` : réduire à une ligne

Combien de filles ont été nommées Khaleesi, au total ?

filter(babynames, name == “Khaleesi”, sex == “F”)

year	sex	name	n	prop
2011	F	Khaleesi	28	0.0000145
2012	F	Khaleesi	146	0.0000754
2013	F	Khaleesi	243	0.000126
2014	F	Khaleesi	369	0.000189
2015	F	Khaleesi	341	0.000175
2016	F	Khaleesi	371	0.000192
2017	F	Khaleesi	466	0.000249

`summarise()` : réduire à une ligne

7 lignes

year	sex	name	n	prop
2011	F	Khaleesi	28	0.0000145
2012	F	Khaleesi	146	0.0000754
2013	F	Khaleesi	243	0.000126
2014	F	Khaleesi	369	0.000189
2015	F	Khaleesi	341	0.000175
2016	F	Khaleesi	371	0.000192
2017	F	Khaleesi	466	0.000249

une seule ligne

total
1964

```
babynames |>
  filter(name == "Khaleesi", sex == "F") |>
  summarise(total = sum(n))
```

Et pour chaque prénom ? Il faudrait découper le tableau par prénom, puis résumer chaque morceau : c'est `group_by()`.

Les fonctions de résumé

Elles prennent un **vecteur** et renvoient **une seule valeur** :

Fonction	Renvoie
`n()`	le nombre de **lignes**
`n_distinct(x)`	le nombre de valeurs **différentes**
`sum(x)`, `mean(x)`, `median(x)`, `sd(x)`	somme, moyenne, médiane, écart type
`min(x)`, `max(x)`	minimum, maximum
`first(x)`, `last(x)`	première, dernière valeur

> Sans `na.rm = TRUE`, un seul `NA` donne un résultat `NA`.

Les fonctions de résumé : l'aide-mémoire

Summary Functions

TO USE WITH SUMMARISE ()

summarise() applies summary functions to columns to create a new table. Summary functions take vectors as input and return single values as output.

summary function

COUNTS

dplyr::**n()** - number of values/rows
dplyr::**n_distinct()** - # of uniques
sum(!is.na()) - # of non-NA's

LOCATION

mean() - mean, also **mean(!is.na())**
median() - median

LOGICALS

mean() - Proportion of TRUE's
sum() - # of TRUE's

POSITION/ORDER

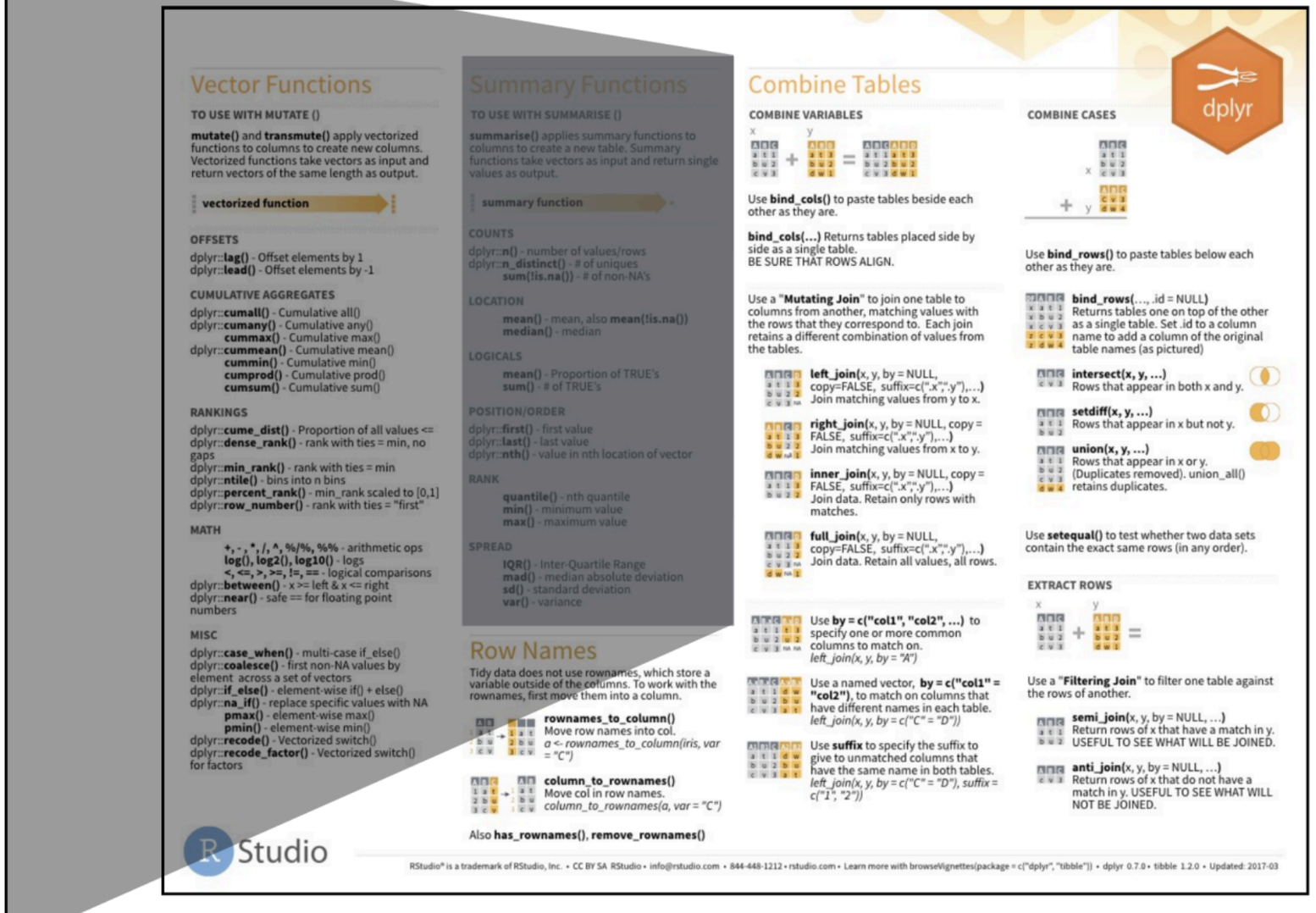
dplyr::**first()** - first value
dplyr::**last()** - last value
dplyr::**nth()** - value in nth location of vector

RANK

quantile() - nth quantile
min() - minimum value
max() - maximum value

SPREAD

IQR() - Inter-Quartile Range
mad() - median absolute deviation
sd() - standard deviation
var() - variance



https://raw.githubusercontent.com/rstudio/cheatsheets/main/data-transformation.pdf

`group_by()` + `summarise()`

pollution : particules fines ($\mu g/m^3$)

city	size	amount
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

```
pollution |>
  group_by(city)
```

`group_by()` + `summarise()`

city	size	amount
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

```
pollution |>
  group_by(city)            # découpe en groupes, ne change pas les données
```

`group_by()` + `summarise()`

city	size	amount
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

→

city	mean	sum	n
New York	18.5	37	2
London	19	38	2
Beijing	88.5	177	2

```
pollution |>
  group_by(city) |>
  summarise(mean = mean(amount), sum = sum(amount), n = n())    # s'applique à chaqu
```

`group_by()` + `summarise()`

une ligne par groupe

city	mean	sum	n
New York	18.5	37	2
London	19	38	2
Beijing	88.5	177	2

```
pollution |>
  group_by(city) |>
  summarise(mean = mean(amount), sum = sum(amount), n = n())
```

Même résultat sans `group_by()` : `summarise(..., .by = city)`. Et avec `group_by(city, size)` : une ligne par combinaison ville × taille.

Sur les vraies données

Combien de filles et de garçons nés aux États-Unis de 1880 à 2017 ?

```
bebes |>
  group_by(sex) |>
  summarise(total = sum(n))
#> # A tibble: 2 × 2
#>   sex       total
#>   <chr>     <dbl>
#> 1 F     172371079
#> 2 M     175749438
```

`ungroup()` retire le groupement :

```
bebes |>
  group_by(sex) |>
  ungroup() |>
  summarise(total = sum(n))
#> # A tibble: 1 × 1
#>       total
#>       <dbl>
#> 1 348120517
```

Compter : `count()`

pollution

city	size
New York	large
New York	small
London	large
London	small
Beijing	large
Beijing	small



count(pollution, city)

city	n
Beijing	2
London	2
New York	2

`count(x)` est un raccourci de `group_by(x) |> summarise(n = n())`. Il compte des **lignes** : toujours se demander ce que représente une ligne (une mesure ? un prénom ? un patient ?).

Attention : le groupement persiste

pollution : 6 lignes, une par ville × taille

city	size	amount
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

```
parts <- pollution |>
  group_by(city, size) |>
  summarise(amount = sum(amount))
```

Attention : le groupement persiste

city	size	amount
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

```
parts <- pollution |>
  group_by(city, size) |>
  summarise(amount = sum(amount))
#> `summarise()` has grouped output by 'city'. You can override using the
#> `.groups` argument.
```

Après `summarise()`, le résultat **reste groupé par `city`** : seule la dernière variable de groupement est retirée.

Attention : le groupement persiste

city	size	amount	part
New York	large	23	0.62
New York	small	14	0.38
London	large	22	0.58
London	small	16	0.42
Beijing	large	121	0.68
Beijing	small	56	0.32

```
parts |> mutate(part = amount / sum(amount))      # sum(amount) est calculé dans cha
```

Les parts font 1 **dans chaque ville** : c'est la part de chaque taille dans sa ville, pas dans l'ensemble des mesures. **Solutions** : `summarise(..., .groups = "drop")`, `ungroup()`, `count()` ou `.by`, qui ne laissent aucun groupement.

✋ À vous : `mutate()` ou `summarise()` ?

Combien de lignes et de colonnes, et que contient `m` ?

1. `echantillons |> group_by(traitement) |> summarise(m = mean(MYC))`
2. `echantillons |> group_by(traitement) |> mutate(m = mean(MYC))`
3. `echantillons |> summarise(m = mean(TP53))`

```
echantillons |>
  group_by(traitement) |>
  summarise(m = mean(MYC))
#> # A tibble: 2 × 2
#>   traitement     m
#>   <chr>      <dbl>
#> 1 ctrl        6.77
#> 2 trt         4.1
echantillons |>
  summarise(m = mean(TP53))
#> # A tibble: 1 × 1
#>       m
#>   <dbl>
#> 1    NA
```

```
echantillons |>
  group_by(traitement) |>
  mutate(m = mean(MYC)) |>
  ungroup() |> select(id, traitement, m)
#> # A tibble: 6 × 3
#>   id    traitement     m
#>   <chr> <chr>      <dbl>
#> 1 S1    ctrl        6.77
#> 2 S2    trt         4.1
#> 3 S3    ctrl        6.77
#> 4 S4    trt         4.1
#> 5 S5    ctrl        6.77
#> 6 S6    trt         4.1
```

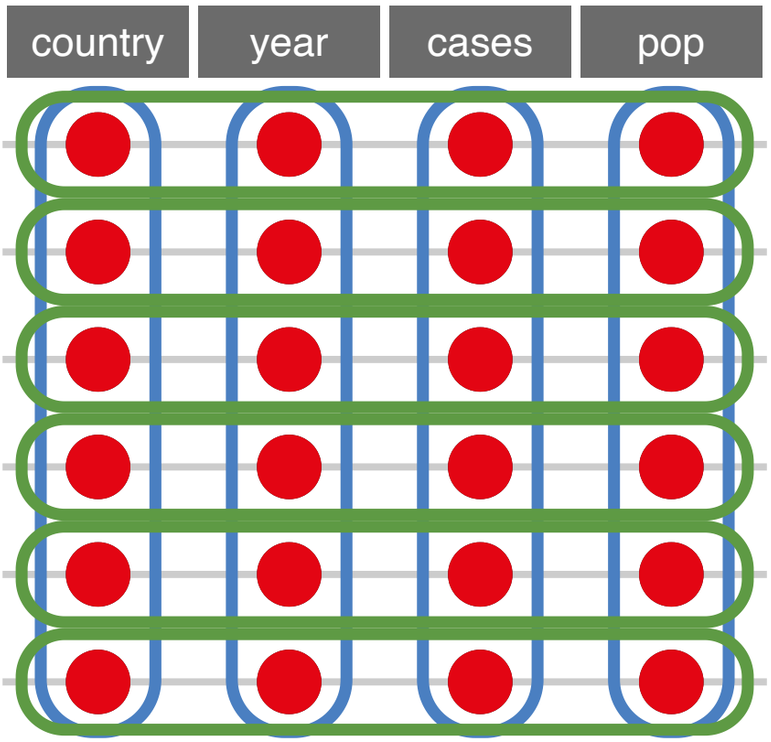
1. `summarise()` : **une ligne par groupe** (2 × 2). **2.** `mutate()` : **autant de lignes qu'avant** (6), la valeur du groupe répétée. **3.** `NA` : il manque `na.rm = TRUE`.

6

DES DONNÉES *TIDY*



Des données *tidy*



Un tableau est *tidy* (« bien rangé ») si :

1. chaque **variable** est dans sa propre **colonne**
2. chaque **observation** est dans sa propre **ligne**
3. chaque **valeur** est dans sa propre **case**

Le package `tidyr` range les données : format **large** → **long** avec `pivot_longer()`, format **long** → **large** avec `pivot_wider()`.

Utile parce que...

table1

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Chaque variable est une colonne, donc un **vecteur** : les opérations de R s'appliquent directement.

Utile parce que...

table1 avec une colonne de plus

country	year	cases	population	rate
Afghanistan	1999	745	19987071	0.0000373
Afghanistan	2000	2666	20595360	0.000129
Brazil	1999	37737	172006362	0.000219
Brazil	2000	80488	174504898	0.000461
China	1999	212258	1272915272	0.000167
China	2000	213766	1280428583	0.000167

```
table1 |> mutate(rate = cases / population)      # en R de base : table1$cases / table1$population
```

Un calcul **vectorisé**, ligne par ligne, sans boucle.

Exemple de données *tidy*

table1

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583



Une colonne par variable (pays, année, cas, population), une ligne par observation (un pays une année).

Exemple de données non *tidy*

table2

country	year	type	count
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583



La colonne `count` mélange **deux variables** (cas et population), selon la valeur de `type` : une observation est répartie sur **deux lignes**.

✋ À vous : ce tableau est-il *tidy* ?

Les **mêmes données** que `table1`, rangées autrement :

table3

country	year	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

✋ À vous : ce tableau est-il *tidy* ? Non.

table3

country	year	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

Deux valeurs dans une même case : `rate` contient les cas *et* la population, sous forme de texte.

Réparer : `separate_wider_delim()`

une valeur par case

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

```
table3 |> separate_wider_delim(rate, delim = "/", names = c("cases", "population"))
```

Piège : `cases` et `population` sont encore du **texte** (`<chr>`), car elles viennent d'une chaîne de caractères. Tout calcul échouera. Il faut les convertir : `mutate(across(c(cases, population), as.numeric))`.

✋ Ranger des données : à vous !

Le tableau `cases` donne un nombre de cas par pays et par année.

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

1. Quelles sont les **variables** de ce tableau ?
2. Dessinez sur papier la version *tidy* : combien de lignes, combien de colonnes ?

03:00

Trois variables : le **pays**, l'**année** et le **nombre de cas**. Les années `2011`, `2012`, `2013` sont des **valeurs** de la variable « année » utilisées comme noms de colonnes. Version *tidy* : 3 pays × 3 années = **9 lignes**, 3 colonnes.

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n

Chaque case du tableau large devient **une ligne** : le pays, l'année (ancien nom de colonne) et la valeur.

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000
FR	2013	7000

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000
FR	2013	7000
DE	2013	6200

Ranger pas à pas

cases

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000



version tidy

country	year	n
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000
FR	2013	7000
DE	2013	6200
US	2013	13000

9 lignes : c'est exactement ce que fait `pivot_longer()`, en une commande.

`pivot_longer()` : du large au long

```
cases |> pivot_longer(cols = 2:4, names_to = “year”, values_to = “n”)
```

le tableau à ranger

les colonnes à empiler (positions, noms, ou « toutes sauf »)

nouvelle colonne : reçoit les anciens noms de colonnes

nouvelle colonne : reçoit les valeurs des cellules

Trois façons de désigner les mêmes colonnes : par **positions**, par **noms**, ou « **toutes sauf** » :

```
cases |> pivot_longer(cols = 2:4, names_to = "year", values_to = "n")
cases |> pivot_longer(cols = c(`2011`, `2012`, `2013`), names_to = "year", values_to = "n")
cases |> pivot_longer(cols = -country, names_to = "year", values_to = "n")
```

Un nom qui commence par un chiffre s’écrit entre accents graves : `` `2011` ``.

`pivot_longer()` : du large au long

format large : une colonne par année

country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

```
cases |> pivot_longer(cols = -country, names_to = "year", values_to = "n")
```

`pivot_longer()` : du large au long

format long : une ligne par pays × année

country	year	n
FR	2011	7000
FR	2012	6900
FR	2013	7000
DE	2011	5800
DE	2012	6000
DE	2013	6200
US	2011	15000
US	2012	14000
US	2013	13000

```
cases |> pivot_longer(cols = -country, names_to = "year", values_to = "n")
```

On peut maintenant résumer **par année** : `cases_long |> summarise(total = sum(n), .by = year)`. C'est aussi le format attendu par `ggplot2`.

`pivot_wider()` : du long au large

Dans `pollution`, quelles sont les variables ? La ville, la quantité de **grosses** particules, la quantité de **petites** particules.

pollution

city	size	amount
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

```
pollution |> pivot_wider(names_from = size, values_from = amount)
```

`pivot_wider()` : du long au large

une ligne par ville, une colonne par taille

city	large	small
New York	23	14
London	22	16
Beijing	121	56

```
pollution |> pivot_wider(names_from = size, values_from = amount)
```

le tableau à remodeler

la colonne qui fournit les noms des nouvelles colonnes

la colonne qui fournit les valeurs des nouvelles cellules

On peut maintenant calculer **entre colonnes**, ligne par ligne : `mutate(ratio = small / large)`. Le format large sert à **présenter** (un tableau d'article) ; le format long sert à **résumer** et à **visualiser**.

✋ À vous : la forme du tableau

Un tableau `comptes` contient 3 gènes (lignes) et 4 échantillons (colonnes `S1` à `S4`), plus une colonne `gene`.

1. Combien de lignes et de colonnes après `pivot_longer(comptes, -gene, names_to = "echantillon", values_to = "n")` ?
2. Et si on oubliait le `-gene` : `pivot_longer(comptes, everything(), ...)` ?

```
comptes <- tibble(gene = c("TP53", "MYC", "BRCA1"), S1 = 1:3, S2 = 4:6, S3 = 7:9, S4 = 10:12)
pivot_longer(comptes, -gene, names_to = "echantillon", values_to = "n") |> dim()
#> [1] 12  3
pivot_longer(comptes, everything(), names_to = "echantillon", values_to = "n")
#> Error in `pivot_longer()`:
#> ! Can't combine `gene` <character> and `S1` <integer>.
```

1. 3 × 4 = **12 lignes** et 3 colonnes (`gene`, `echantillon`, `n`).
2. Une **erreur** : on tente d'empiler du texte (`gene`) et des nombres dans la même colonne `n`.

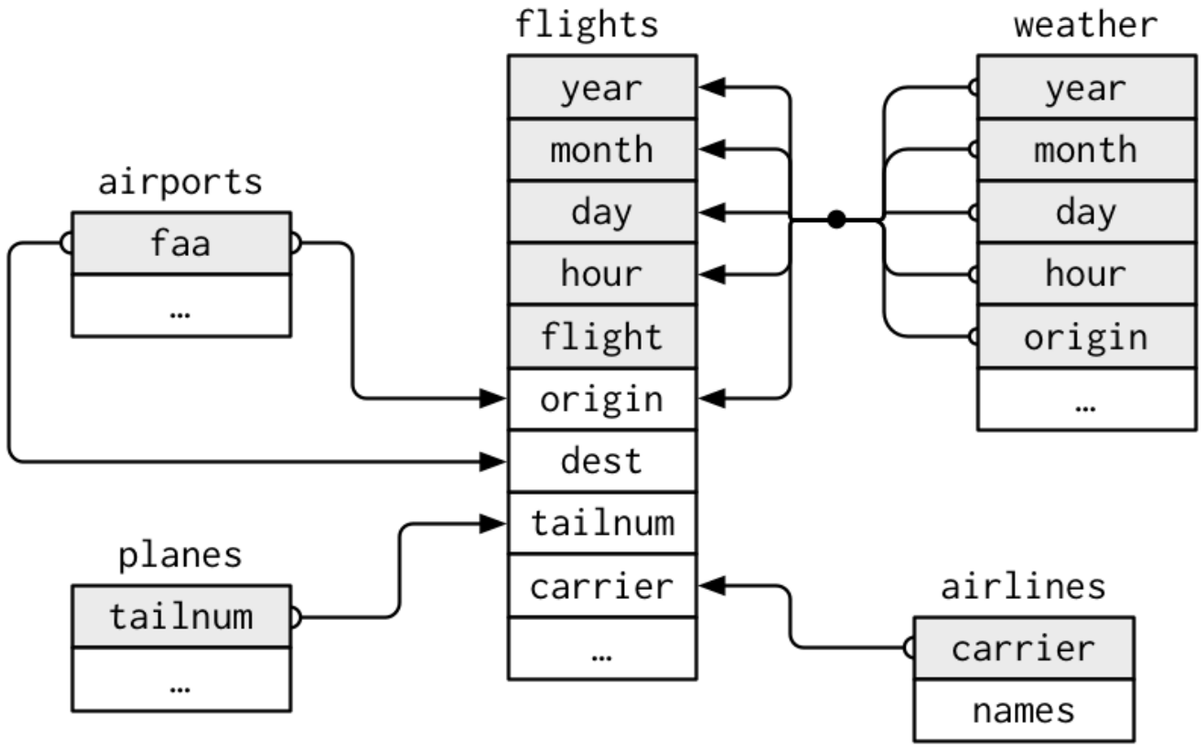
TP2

7

LES JOINTURES



Des données *tidy* réparties dans plusieurs tableaux



`nycflights13` : vols au départ de New York en 2013 (TP3)

Chaque tableau décrit **une** chose : les vols, les compagnies, les aéroports, les avions, la météo.

Ils sont reliés par des colonnes communes : les **clés** (`carrier`, `faa`/`dest`, `tailnum`…).

Quelles compagnies ont les plus longs retards ?

`flights` contient les retards, mais seulement le **code** de la compagnie (`UA`, `AA`…). Les **noms** sont dans `airlines`.

Il faut **joindre** les deux tableaux par la clé `carrier`.

Joindre deux tableaux

```
band |> left_join(instrument, by = “name”)
```

le tableau de gauche : celui qu’on complète

le tableau de droite : celui qui apporte des colonnes

la clé : la colonne commune aux deux tableaux

Si la clé ne porte pas le même nom des deux côtés : `by = join_by(dest == faa)`.

Les jointures

`band |> left_join(instrument, by = "name")` : **toutes** les lignes de `band`, complétées par `instrument`

band

name	band
Mick	Stones
John	Beatles
Paul	Beatles

instrument

name	plays
John	guitar
Paul	bass
Keith	guitar

Mick ne joue d'aucun instrument connu : NA

name	band	plays
Mick	Stones	**NA**
John	Beatles	guitar
Paul	Beatles	bass

Les jointures

`band |> right_join(instrument, by = "name")` : **toutes** les lignes de `instrument`, complétées par `band`

band

name	band
Mick	Stones
John	Beatles
Paul	Beatles

instrument

name	plays
John	guitar
Paul	bass
Keith	guitar

Keith n'a pas de groupe connu : NA

name	band	plays
John	Beatles	guitar
Paul	Beatles	bass
Keith	**NA**	guitar

Les jointures

`band |> inner_join(instrument, by = "name")` : seulement les lignes présentes **dans les deux**

band

name	band
Mick	Stones
John	Beatles
Paul	Beatles

instrument

name	plays
John	guitar
Paul	bass
Keith	guitar

Mick et Keith disparaissent

name	band	plays
John	Beatles	guitar
Paul	Beatles	bass

Les jointures

`band |> full_join(instrument, by = "name")` : **toutes** les lignes des deux

band

name	band
Mick	Stones
John	Beatles
Paul	Beatles

instrument

name	plays
John	guitar
Paul	bass
Keith	guitar

des NA de chaque côté

name	band	plays
Mick	Stones	**NA**
John	Beatles	guitar
Paul	Beatles	bass
Keith	**NA**	guitar

Les jointures

`band |> anti_join(instrument, by = "name")` : les lignes de `band` **sans correspondance** dans `instrument`

band

name	band
Mick	Stones
John	Beatles
Paul	Beatles

instrument

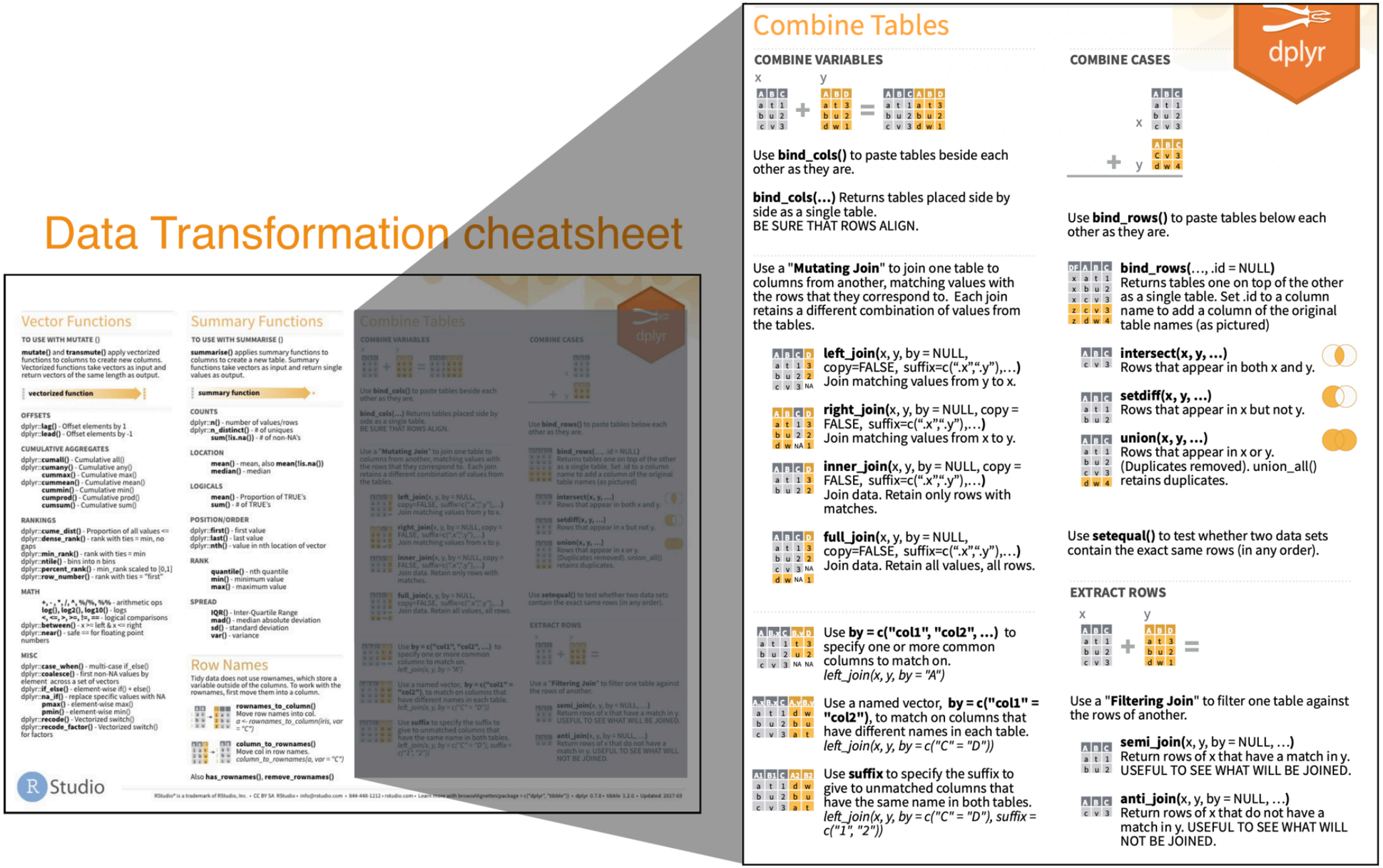
name	plays
John	guitar
Paul	bass
Keith	guitar

l'outil de vérification : qui n'a pas trouvé de partenaire ?

name	band
Mick	Stones

`left_join()` est la plus utilisée. `anti_join()` ne sert pas à construire un résultat, mais à **vérifier** une jointure. `semi_join()` garde les lignes de `x` qui ont une correspondance, sans ajouter de colonnes.

Les jointures : l'aide-mémoire



https://raw.githubusercontent.com/rstudio/cheatsheets/main/data-transformation.pdf

Quand la clé pose problème

echantillons

id	TP53
S1	5.1
S2	9.7
S3	4.8
S4	**NA**
S5	6.2
S6	10.9

lots

id	lot
S1	L1
S2	L1
S3	L2
S3	L2
~~s5~~	~~L3~~
S9	L3

`left_join(echantillons, lots, by = "id")` : combien de lignes ?

Quand la clé pose problème

7 lignes au lieu de 6

id	TP53	lot
S1	5.1	L1
S2	9.7	L1
S3	4.8	L2
S3	4.8	L2
S4	**NA**	**NA**
~~S5~~	~~6.2~~	~~**NA**~~
S6	10.9	**NA**

```
left_join(echantillons, lots, by = "id")
```

- S3 en double dans `lots` : sa ligne est **dupliquée**
- `s5` ≠ `S5` (casse) : S5 reste **sans lot**
- S4, S6 absents de `lots` : `NA`

Aucun message.

Vérifier une jointure

Trois vérifications, à faire **à chaque jointure** :

```
nrow(echantillons)                                          # 1. lignes avant…
#> [1] 6
echantillons |> left_join(lots, by = "id") |> nrow()       #    … et après : 7 ≠ 6
#> [1] 7
lots |> count(id) |> filter(n > 1)                          # 2. clés en double
#> # A tibble: 1 × 2
#>   id        n
#>   <chr> <int>
#> 1 S3        2
echantillons |> anti_join(lots, by = "id") |> pull(id)      # 3. clés sans correspondance
#> [1] "S4" "S5" "S6"
```

Une jointure **à gauche** ne doit pas changer le nombre de lignes. Si c'est le cas, une clé est en double dans le tableau de droite. Pour l'exiger : `left_join(..., relationship = "many-to-one")`, qui produit alors une erreur.

✋ À vous : combien de lignes ?

Avec `echantillons` (6 lignes, S1 à S6) et `lots` (S1, S2, S3, S3, s5, S9), combien de lignes renvoie :

1. `inner_join(echantillons, lots, by = "id")`
2. `full_join(echantillons, lots, by = "id")`
3. `anti_join(lots, echantillons, by = "id")`
4. `semi_join(echantillons, lots, by = "id")`

```
inner_join(echantillons, lots, by = "id") |> pull(id)
#> [1] "S1" "S2" "S3" "S3"
full_join(echantillons, lots, by = "id") |> pull(id)
#> [1] "S1" "S2" "S3" "S3" "S4" "S5" "S6" "s5" "S9"
anti_join(lots, echantillons, by = "id") |> pull(id)
#> [1] "s5" "S9"
semi_join(echantillons, lots, by = "id") |> pull(id)
#> [1] "S1" "S2" "S3"
```

`semi_join()` garde les lignes de `x` qui ont une correspondance, **sans les dupliquer** ni ajouter de colonnes.

TP3

8

DE R DE BASE AU *TIDYVERSE*

✋ À vous : traduire le script de la semaine dernière

Le script **corrigé** de l'audit TP53 (module 5), en R de base. Réécrivez chaque bloc avec les verbes du *tidyverse*.

```
donnees <- read.table("/home/public/R/patients.tsv", header = TRUE, sep = "\t")
donnees$groupe <- tolower(donnees$groupe)

# 1. Différence d'expression moyenne entre les groupes
moy_traite   <- mean(donnees$TP53[donnees$groupe == "traite"], na.rm = TRUE)
moy_controle <- mean(donnees$TP53[donnees$groupe == "controle"], na.rm = TRUE)
moy_traite - moy_controle

# 2. Nombre de patients avec une expression > 8
sum(donnees$TP53 > 8, na.rm = TRUE)

# 3. Les 3 patients avec la plus forte expression
head(donnees[order(donnees$TP53, decreasing = TRUE), ], 3)
```

Traduction en *tidyverse*

```r
donnees <-
  read_tsv("/home/public/R/patients.tsv") |>
  mutate(groupe = str_to_lower(groupe))
```

```r
# 1. une moyenne par groupe, puis la différence
donnees |>
  summarise(moyenne = mean(TP53, na.rm = TRUE),
            .by = groupe) |>
  pivot_wider(names_from = groupe,
              values_from = moyenne) |>
  mutate(difference = traite - controle)
#> # A tibble: 1 × 3
#>   traite controle difference
#>    <dbl>    <dbl>      <dbl>
#> 1   9.94     5.48       4.46
```

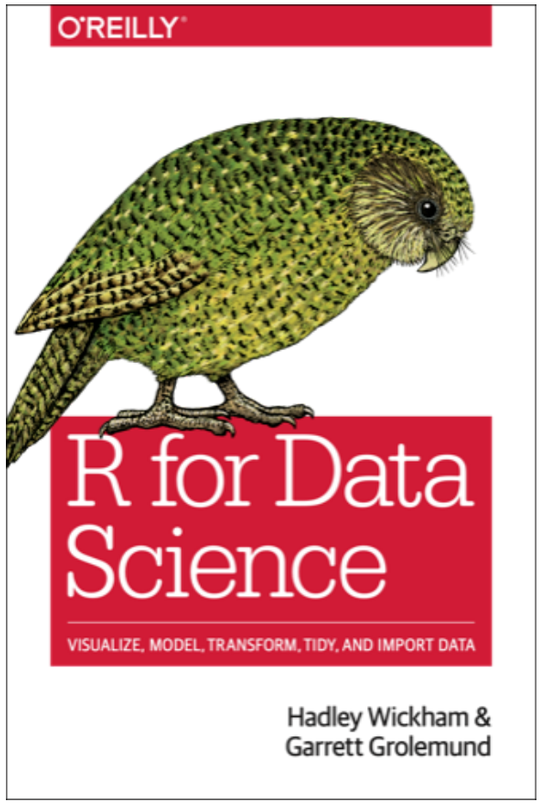
```r
# 2. filter() écarte déjà le NA de P05
donnees |> filter(TP53 > 8) |> nrow()
#> [1] 5
```

```r
# 3. pas de piège sort() / order()
donnees |> slice_max(TP53, n = 3)
#> # A tibble: 3 × 4
#>   id    groupe   age  TP53
#>   <chr> <chr>  <dbl> <dbl>
#> 1 P03   traite    47  11.6
#> 2 P07   traite    41  10.9
#> 3 P01   traite    34   9.7
```

Mêmes résultats que la semaine dernière (4.46 ; 5 ; P03, P07, P01). Deux des trois erreurs de l'audit disparaissent d'elles-mêmes : `filter()` ne garde que les `TRUE`, et `slice_max()` ne confond pas valeurs et positions. La casse, elle, reste à corriger à la main.

Ressources

Ce module a été construit à partir de ressources libres de **RStudio (Posit)** :



- *Master the Tidyverse* (G. Grolemund, RStudio Education, CC BY-SA 4.0) : exemples (*babynames*, *pollution*, *cases*, *band / instrument*), jeux de données des TP (*nycflights13*) et principe des tableaux animés, repris de la version PowerPoint 2023 du cours
- *R for Data Science*, 2e édition (H. Wickham, M. Çetinkaya-Rundel, G. Grolemund) : https://r4ds.hadley.nz
- *Introduction to the Tidyverse* (O. Gimenez, *Reproducible science workshop*) : exemple `table3` de `tidyr` (deux valeurs dans une case)
- Aide-mémoire *Data transformation with dplyr* et *Data tidying with tidyr* : https://posit.co/resources/cheatsheets/

Pour aller plus loin : les chapitres *Data transformation*, *Data tidying* et *Joins* de *R for Data Science*.